

RAP 後端開發練習 4：Service Definition / Service Binding 與發布流程

Lecture

這一課要做什麼

rap01 ~ rap03 把 RAP 五層架構的前三層 (Table → CDS Interface View → Behavior Definition) 都摸過了，這一課接上第四、第五層：**Service Definition** (把一個或多個 CDS View 包裝成一個服務) 跟 **Service Binding** (把這個服務發布成真正可以用 HTTP 呼叫的 OData 端點)。

這一課也是課程分工原則正式落地的第一課 (見 README「教學分工原則」)：

- **Service Definition (ZRAP04_SD)**：Claude 已經建立並啟用——這個物件有完整的 ADT REST API，技術上沒有理由要你手動建。
- **Service Binding (ZRAP04_SB)**：換你在 Eclipse ADT 動手建立 + Publish——這不是分工政策選擇性地要你做，是技術上的硬性限制：已經實測證實，用 ADT REST API 手動 POST 建出來的 Service Binding，會缺少 Eclipse 精靈才會觸發的一段後端註冊步驟，導致 Publish 永遠失敗、而且錯誤訊息完全看不出真正原因 ([.claude/rules/sap-adt-mcp.md](#) 第 40.9 節有完整的踩坑記錄)。下方會給你一步一步的 Eclipse 操作指引。

Service Definition：把 CDS View 包裝成一個服務

語法很單純，看完整範例之前先認識兩個語法元素：

- **define service <名稱> { ... }**：一個 Service Definition 可以 **expose** 多個 CDS View，全部會出現在同一個 OData 服務底下，各自是一個獨立的 Entity Set。
- **expose <CDS View> as <別名>**：**as** 後面的別名就是這個實體在 OData 服務裡的名字 (Entity Set 名稱)——不寫 **as** 的話會直接用 CDS View 的技術名稱，但那通常太長也不好看，實務上都會取一個簡短別名。

這是這一課完整的 Service Definition：

```
@EndUserText.label: 'RAP04 Managed vs Unmanaged Demo'
define service ZRAP04_SD {
  expose ZI_RAP02_TASK as TaskManaged;
  expose ZI_RAP03_UMTEST as TestUnmanaged;
}
```

這一課刻意把 rap02 的 **Managed CDS View (ZI_RAP02_TASK)** 跟 rap03 的 **Unmanaged CDS View (ZI_RAP03_UMTEST)** 包進同一個服務，別名分別叫 **TaskManaged / TestUnmanaged**——這樣你之後在同一個 Fiori Elements 預覽畫面裡就能直接對照「同一個服務底下，Managed 實體讀取正常但寫入會 Dump，Unmanaged 實體讀寫都正常」這個第 43 / 44 節記錄過的關鍵限制，不用開兩個獨立的服務來回切換。

Eclipse ADT 建立 Service Definition：Step by Step

ZRAP04_SD 是 Claude 用 ADT API 建的 (這個物件有完整 REST API，技術上沒有理由要你手動建)，但精靈操作本身很簡單，值得認識一次 (查證 SAP 官方 openSAP RAP 課程教材步驟)：

- 1. 對著要包裝的 CDS View (例如 ZI_RAP02_TASK) 按右鍵 → 選單裡直接有 **New Service Definition** (這系統一貫的右鍵直接捷徑模式，不用繞 Other ABAP Repository Object)。
- 2. 填 **Name** (例如自己練習用的名稱)、**Description**、**Project / Package** 自動帶出，按 **Next**。
- 3. 選傳輸請求 (\$TMP 直接 **Finish**，不用選——注意：官方教材這裡精靈流程是 Next → Next → Finish 三步，中間會先跳「選擇 Template」畫面，見下一步)。
- 4. 選 **Template**：畫面上通常只有一個選項 **Define Service**，選了按 **Finish**。
- 5. 精靈生成的骨架不是空白檔案，而是已經寫好 `define service <名稱> { expose <一個猜測的 CDS View> as <別名>; }` 這種「示範用的假 expose」——把它改成你真正想要 expose 的 CDS View (可以一次 expose 多個，像這一課的 ZRAP04_SD 一樣)。
- 6. 存檔 (**Ctrl+S**) + 啟用 (**Activate**)。

練習：自己建一個 Service Definition

輪到你了：把上面 rap03 練習建的 BDEF (不管你選的是①Managed 練習還是②Unmanaged 挑戰) 對應的 CDS View，用上面的精靈步驟包成一個你自己的 Service Definition (`expose <你的 CDS View> as <自訂別名>;`)。如果你想順便看到畫面效果，可以繼續照下面「Eclipse ADT 操作步驟：建立並發布 Service Binding」的同一套流程，幫這個練習用的 Service Definition 也建一個 Service Binding + Publish + Preview——步驟完全一樣，只是這次名稱換成你自己的物件。完成後跟我說一下建立過程跟最終狀態 (啟用成功與否、Preview 看到的畫面)，我會幫你核對。

Service Binding：讓服務真正「活起來」

Service Definition 只是一份「這個服務裡有哪些實體」的宣告，本身不能被外部呼叫。要讓它變成一個真正能打 GET/POST 的 HTTP 端點，需要 **Service Binding**：指定要用哪種協定 (本課用 **OData V2 - UI**，這是目前這系統驗證過確實能跑 Fiori Elements 預覽的組合；V4 概念上也存在，但這系統偏舊、沒有特別驗證過 V4 的完整發布流程，不強求)。Service Binding 建立後還要按一個 **Publish** 按鈕，系統才會真的在 ICF (Internet Communication Framework，管理這個系統所有 HTTP 端點的地方) 底下掛一個新節點——這一步之前也在 REST 課程學過類似概念 (**SICF**)，只是 RAP 這邊是透過 Eclipse 精靈自動完成 ICF 註冊，不需要自己手動開 **SICF** 交易碼設定。

****Binding Type** 除了協定版本 (V2 / V4) 之外，還有一個「UI」跟「Web API」的分類要選，兩者差異不小 (查證 SAP 官方文件 *Service Binding* 頁面確認) **：

	OData V2 - UI	OData V2 - Web API
用途	讓 SAP Fiori Elements 或其他 UI 客戶端能接上這個服務	給「UI 以外」的所有用途用，可以被不特定的消費者 (unknown consumer) 透過 OData 呼叫
\$metadata 內容	含 UI 專屬資訊 (標籤、Value Help、Side Effects.....這些就是從 rap02 教的 @UI.* Annotation / Metadata Extension 轉譯進去的)	不含任何 UI 專屬資訊 ，是一份純技術性的 OData 服務
Eclipse ADT 的 Preview 功能	可以直接開出 Fiori Elements 畫面	官方文件明講「Fiori Elements preview: Not available」，沒有這個

設計定位

給你自己的 Fiori 前端用

給外部系統整合、第三方消費，官方建議命名字首用 **API_** (UI 服務對應用 **UI_**)

這門課一律選 **UI**：因為我們是靠 Eclipse 的 **Preview** 按鈕直接看到 Fiori Elements 畫面來驗證資料對不對，這個功能只有 UI 類型才有；如果選 Web API，Publish 之後只能拿 URL 去 Postman / 瀏覽器打純 JSON，看不到渲染出來的畫面，也看不到 rap02 教的 **@UI.*** Annotation 有沒有生效。如果之後你自己的專案裡，這個 OData 服務的目的是給別的系統呼叫（不是你自己的 Fiori 應用），才該選 Web API。

⚠ 容易誤解的地方：「**Postman / 任何 OData 客戶端都能呼叫 UI 類型**」不代表兩者沒差——rap08 用 Postman 直接打過 **ZRAP08_SB** (UI 類型) 完全成功，這證實了「能不能被外部工具呼叫」本身不是 UI 跟 Web API 的差異點，兩者底層都是標準 OData V2 協定，語法、CRUD 操作完全一樣。真正的差異在**定位與治理**，不是技術呼叫能力：

- **穩定性承諾 (Release Contract)**：官方文件的 OData Exposure Comparison 表格顯示，只有 **Web API** 類型能在 ADT 右鍵選單走「API State」流程正式**釋出成受保管的 API 契約** (C1 / C2 Contract) ——一旦釋出，SAP 承諾這個服務的欄位、行為在未來版本升級**不會無預警破壞相容性**。UI 類型沒有這個釋出機制，設計上就是「跟著你的 Fiori 畫面走」，你隨時可能為了改善畫面調整欄位，SAP 不會（也不該）保證它長期穩定。如果有另一個系統要長期依賴你的服務做整合，用 UI 類型技術上「能動」，但沒有任何穩定性保證，你哪天調整了 CDS View，對方可能就悄悄壞掉、沒人事先警告。
- **\$metadata** 精簡，減少無關雜訊：同一份文件表格顯示，**UI Service V2** 會自動多帶 **SAP__Currencies / SAP__UnitsOfMeasure** 這兩個內建 **Entity Set** (給 Fiori 畫面的幣別/單位下拉選單用)，**Web API V2** 完全不會有——外部系統整合用不到「畫面下拉選單」這種概念，多帶出來只是雜訊。
- **命名慣例是治理信號**：官方建議 Web API 用 **API_** 字首、UI 服務用 **UI_** 字首，讓團隊一眼分得出「這是給外部系統依賴的正式契約」還是「這是跟著某個 Fiori App 走的內部實作細節」，避免有人誤把 UI 服務當穩定 API 拿去給別的系統整合。

一句話：即使 Postman 兩邊都打得通，UI 類型是「支撐我自己的 Fiori 前端」、Web API 是「讓另一套系統長期穩定依賴」，兩者的差異是**承諾等級**，不是技術能力。

⚠ 為什麼 **Service Binding** 一定要用 **Eclipse 精靈**，不能用 **ADT API 手動建**？已經實測過：直接用 curl 對 **POST /sap/bc/adt/businessservices/bindings** 送一份格式完全正確、Schema 完全對的 XML，物件確實會被建立、也能啟用成 Active，但不管怎麼重試 Publish，Gateway 永遠回報 **Service Definition is not available (SDDIC_ADT_SRV011)** ——因為 Eclipse 精靈在建立 Service Binding 的過程中，除了寫入這個 DDIC 物件本身，還會額外觸發一個只有精靈流程才會做的背景步驟：把 Service Definition 的 Metadata 編譯、登錄進 Gateway 執行期的模型快取。手動 API 建立完全跳過了這一步，事後也沒有任何方法補救，只能整個物件重新用精靈建一次。

Eclipse ADT 操作步驟：建立並發布 Service Binding

✅ 以下步驟已由使用者實際操作過一次並截圖記錄 (**zrap04_ECLIPSE_SERVICE_BINDING_操作記錄.docx**)，內容已依實際畫面校正過：

1. 在 **Project Explorer** (專案總管) 展開你的 ABAP Project，找到剛剛 (Claude 已建好的) **ZRAP04_SD** (Service Definition，在 Package **\$TMP** 底下)。

2. 對著 **ZRAP04_SD** 按滑鼠右鍵，選單裡直接就有 **New Service Binding** (不需要繞經 **Other ABAP Repository Object** 精靈，這個系統的右鍵選單已經內建這個捷徑) 。
3. 跳出的 **New Service Binding** 對話框一次填齊所有欄位：
 - **Project**：維持預設
 - **Package**：**\$TMP**
 - **Name**：**ZRAP04_SB** (務必跟這個名字一致——OData 技術服務名稱 = Service Binding 物件名稱，Claude 給你的驗證程式跟你自己用瀏覽器/Postman 測試都會用這個名字組網址)
 - **Description**：自由填
 - **Binding Type**：下拉選單選 **OData V2 - UI** (畫面會有一句提示「It's recommended to use OData V4」，忽略即可——這系統版本只支援 V2，V4 不適用)
 - **Service Definition**：確認自動帶出來的是 **ZRAP04_SD**
 - 按 **Next** (不是 Finish，這裡還有一步)
4. ⚠ 接著會跳出「**Select Transport Request**」畫面 (原始講義漏了這一步，已補上)：因為套件是 **\$TMP**，畫面上會顯示提示文字「No change recording enabled for package \$TMP」，代表這個套件不需要傳輸請求——什麼都不用選，直接按 **Finish** 即可。
5. 精靈跑完後會自動開啟 Service Binding 的編輯畫面，此時 **Local Service Endpoint** 顯示 **Unpublished**，右上角有 **Publish** 按鈕。
6. 點 **Publish**。
7. 等待幾秒鐘，**Local Service Endpoint** 狀態會變成 **Published**，右邊會出現 **Service URL** (**/sap/opu/odata/sap/ZRAP04_SB**) 跟 **Entity Set and Association** 清單，列出 **TaskManaged**、**TestUnmanaged** 兩個實體，旁邊有 **Preview...** 按鈕。
8. 點一個 Entity Set (例如先選 **TaskManaged**) → 按 **Preview...**：
 - 瀏覽器會自動開啟一個新分頁，先跳出登入畫面 (帳號密碼)，輸入你平常登入這套系統的帳密
 - 登入後會跳到 Fiori Elements List Report 畫面——已實際驗證：**TaskManaged** 的篩選列跟表格欄位標題正確顯示 **Task ID / Status / Text / Priority / Due Date**，代表 rap02 做的 **@UI.headerInfo / @UI.lineItem / @UI.selectionField** Annotation 確實生效，不是只在 ADT 裡看得到、Gateway 真的有讀懂這些標記
 - 因為底層是 rap02/rap03 的 **Managed** BDEF，列表讀取正常 (不受第 43 節的白名單限制影響)；但如果你嘗試按畫面上的 **Create** 按鈕新增一筆，會直接遇到 **Dump (MESSAGE_TYPE_X_TEXT)**——這是預期中的已知限制，不是你操作錯誤，不用嘗試排除，直接關掉這個錯誤畫面即可
9. 回到 Service Binding 編輯畫面，換選 **TestUnmanaged** → 再按一次 **Preview...**：因為底層是 rap03 的 **Unmanaged** 實作類別，這是這系統上少數幾個真的能端對端動起來的路徑——按 **Create**，隨便填一個 **id** (例如 **TEST000001**，最多 10 碼) 跟 **descr**，存檔應該能成功，畫面會跳出新增的那一行。

用「自我呼叫」驗證：不用等你截圖回報，Claude 也能自動確認

REST 課程 (rs07) 教過一個技巧：ABAP 程式可以用 **cl_http_client=>create_by_destination(destination = 'NONE')** 建立一個「呼叫自己這套系統」的 HTTP Client，不需要知道對外主機名稱 / Port ('NONE' 是 SAP 內建的特殊虛擬目的地，代表「呼叫我自己」)。這一課用同樣的技巧寫了一支驗證程式 **ZR_RAP04_SELFTEST**：

1. **GET TaskManaged** 實體集 (Managed，驗證讀取正常)
2. **GET TestUnmanaged** 實體集 (Unmanaged，建立前的狀態)
3. 用 **X-CSRF-Token: Fetch** 這個 Header 換一個 CSRF Token——這步驟你應該覺得眼熟：跟本課程一路以來 Claude 呼叫 ADT REST API 前要先用同樣手法拿 Token 是同一套機制 (見 **.claude/rules/sap-adt-mcp.md** 第 4 節)，OData 的寫入操作 (**POST/PUT/DELETE**) 一樣要走這個保護機制，任何要真正

呼叫這個服務寫資料的外部程式 (不管是 Postman、別的系統、還是這支自我呼叫程式) 都逃不掉這一步。

- 4. 用拿到的 Token，POST 一筆新資料到 `TestUnmanaged` (走 Unmanaged 路徑，預期成功)
- 5. 最後直接用 `SELECT` 查資料庫，確認這筆資料真的寫進 `ZRAP03_UMTEST` 表格

⚠️ ⚠️ 已更正：這支程式的完整版本在這系統上跑不動，不是等你 **Publish** 就能跑——是這套系統本身的限制。實際測試發現：`programrun` 執行時，程式本身已經佔用了系統有限的 Dialog Work Process，`cl_http_client` 的自我呼叫要再讀一次真正的 RAP Entity 資料，需要再拿到一個空出來的 Work Process，形成自我等待的僵局，最終逾時斷線 (`RFC_CLOSED`) ——已經用查證階段就發布成功、Eclipse Preview 也顯示過資料的既有服務 `ZRAP01_SB3` 做對照測試，一樣卡住，證實不是 `ZRAP04_SB` 或這支程式寫錯，是「自我呼叫讀取真實資料」這個模式在這套小型系統上普遍卡死 (唯獨 `$metadata` 這種輕量請求不受影響)。詳細排查記錄見 `.claude/rules/sap-adt-mcp.md` 第 45 節。

結論：這一課的資料驗證改成完全由你在 **Eclipse Preview** 操作確認 (見上方步驟 10)，`ZR_RAP04_SELFTEST` 保留下來當語法參考 / 示範自我呼叫技巧的寫法，但不要嘗試執行它的完整版本。

小知識點 (查證得來，不是猜的)：這一課建立驗證程式之前，先用一支探測程式打了已經在查證階段發布過的 `ZRAP01_SB3` 服務的 `$metadata`，確認 OData V2 的 Entity 屬性名稱完全沿用 **CDS View** 宣告的欄位名稱、不會自動轉大小寫 (`root_id / descr` 這種小寫欄位，`$metadata` 裡也是小寫)。所以 `ZI_RAP03_UMTEST` 的 `id / descr` 兩個欄位，OData 裡也是原封不動的小寫 `id / descr`，不是 Fiori 常見的 `Id / Descr` 這種帕斯卡命名——這支驗證程式的 JSON payload 已經照這個結論寫好。

學習目標

- 能寫出 Service Definition 語法 (`define service / expose ... as ...`)，知道一個服務可以包裝多個不同來源 (甚至不同 Behavior 類型) 的 CDS View
- 能在 Eclipse ADT 完整走過一次「對著 CDS View 右鍵 → New Service Definition → Next → Transport → 選 Template → Finish」的精靈流程，知道精靈生成的骨架是帶示範用假 `expose` 的模板，不是空白檔案
- 知道 Service Binding 是「讓服務真正能被 HTTP 呼叫」的最後一步，理解 Publish 動作背後對應到 ICF 節點註冊
- 能在 Eclipse ADT 完整走過一次「New Service Binding 精靈 → 選 Binding Type → Finish → Publish → Preview」的操作流程
- 知道為什麼這系統的 Service Binding 一定要用 Eclipse 精靈建立，ADT REST API 手動建會缺一層隱藏的後端註冊步驟，Publish 永遠失敗且錯誤訊息看不出真正原因
- 能在同一個發布出來的 Fiori Elements 預覽畫面裡，對照出「Managed 實體讀取正常、Create 會 Dump」vs「Unmanaged 實體讀寫都正常」的具體差異，並說得出背後原因 (第 43 / 44 節)
- 知道 `cl_http_client=>create_by_destination( destination = 'NONE' )` 這個「呼叫自己系統」的自我呼叫技巧，以及 OData 寫入操作一樣需要先用 `X-CSRF-Token: Fetch` 換 Token 才能送出

物件清單

物件	名稱	型別	建立者	狀態
Service Definition	<code>ZRAP04_SD</code>	<code>SRVD/SRV</code>	Claude (ADT API)	✅ 已建立、已啟用

物件	名稱	型別	建立者	狀態
Service Binding	ZRAP04_SB	SRVB/SVB	你 (Eclipse 精靈)	 已建立並 Publish 成功 (ADT 讀回 <code>srvb:published="true"</code> 確認)，操作過程截圖記錄於 <code>zrap04_ECLIPSE_SERVICE_BINDING_操作記錄.docx</code>
自我呼叫驗證程式	ZR_RAP04_SELFTEST	PROG/P	Claude (ADT API)	 已建立、已啟用。讀取 (GET) 用 SE38 手動執行已驗證成功 (<code>programrun</code> 會卡死，見第 45 節)；寫入 (POST) 在自我呼叫情境下遇到獨立的 CSRF Token 驗證失敗問題，找不到可靠 workaround，保留當語法參考，不當作寫入驗證依據
(查證用 探測程 式，可忽 略)	ZR_RAP04_PROBE / ZR_RAP04_PROBE2	PROG/P	Claude (ADT API)	已建立、已啟用，用來確認 OData 欄位大小寫慣例 + 排查自我呼叫卡住的根因，沒有教學用途
Metadata Extension (收尾補課)	ZI_RAP03_UMTEST	DDLX/EX	Claude (ADT API)	 已建立、已啟用——rap03 建 <code>ZI_RAP03_UMTEST</code> 時沒做 UI Annotation，這一課實測 Create 畫面空白才發現需要補上，順便修正了 <code>zi_rap03_umtest.ddls.abap</code> 補上欄位 <code>@EndUserText.label</code> (見下方驗證方式第 4 點的除錯過程)

全部在 \$TMP 套件，`sap_inactive_objects` 確認 0 筆殘留。

驗證方式

-  **Eclipse** 操作驗證 (已完成，主要驗證管道)：`zrap04_ECLIPSE_SERVICE_BINDING_操作記錄.docx` 完整截圖記錄了實際操作過程，Claude 已逐張確認：
 - Service Binding 建立畫面填寫內容跟講義一致 (`ZRAP04_SB` / `$TMP` / OData V2 - UI / `ZRAP04_SD`)
 - Publish 後 `Local Service Endpoint` 顯示 **Published**，Service URL 正確是 `/sap/opu/odata/sap/ZRAP04_SB`
 - TaskManaged** 的 **Fiori Elements Preview** 實際載入成功，篩選欄位跟表格欄位標題正確顯示 `Task ID / Status / Text / Priority / Due Date`——這連帶驗證了 rap02 的 `@UI.*` Metadata Extension 真的有被 Gateway 讀懂，不是只在 ADT 編輯器裡看得到
 - ADT 讀回 `ZRAP04_SB` 也確認 `srvb:published="true"`，跟畫面顯示一致，雙重確認
-  **programrun** 無頭驗證走不通，但 **SE38** 手動執行可以驗證「讀取」：`ZR_RAP04_SELFTEST` 透過 `programrun` 一律卡死斷線 (`RFC_CLOSED`，系統層級 Work Process 資源限制，見第 45 節)，但改到 **SE38 (F8)** 手動執行，**GET** 讀取完全成功——實測 **TaskManaged** 讀到 200 OK 空清單 (`ZI_RAP02_TASK` 從沒寫過測試資料)、**TestUnmanaged** 讀到 rap03 留下的真實資料列 `UM0001`，證實這系統的自我呼叫技巧「讀取」是可靠的，只是不能透過 `programrun` 執行。

3.   **ZR_RAP04_SELFTEST 的 POST (Create)** 遇到另一個獨立、更難排查的問題：CSRF Token 有正確換到，即使手動把換 Token 回應的 **Set-Cookie** 明確轉發到 **POST** 請求，還是一律 **403 CSRF token validation failed**。合理推測跟這系統可能是多台 Application Server、Gateway 的 CSRF Token 驗證需要 Server 親和性有關（自我呼叫的 GET / POST 兩次獨立請求可能被分派到不同節點），這是比「讀取會卡住」更難排查的問題，已判斷投入產出比過低、不建議繼續深挖（技術細節見 [.claude/rules/sap-adt-mcp.md](#) 第 45 節）。
4.   **最終改用 Eclipse Fiori Elements Preview 完整驗證成功**：對 **TestUnmanaged** 按 **Create**，填入 **id / descr** 後按 **Save**——存檔當下畫面雖然變空白（這系統版本 S/4HANA 1909 的舊版 Fiori Elements，Save 後導轉頁面有顯示上的小毛病，不影響實際功能），但回到 List Report 按 **Go** 重新查詢，**確認新資料真的寫進資料庫**（跟 rap03 EML 測試留下的舊資料 **UM0001** 一起顯示）。這才是這一課「寫入是否真的成功」的最終、可靠的驗證依據。
 - 過程中順便補齊了 **rap03** 遺留的教材缺口：**ZI_RAP03_UMTEST** 建立當時（rap03）沒有做任何 UI Annotation，這一課實測 Create 才發現畫面完全空白 / 沒有欄位標題，追出兩個根因並修正：① **@UI.facet** 這個標記要寫在 **annotate view ... with { }** 區塊裡面（第一個欄位之前），不能放在區塊外面跟 **@UI.headerInfo** 同一層，否則會啟用失敗、報 **wrong position (wrong scope)**；② 表格欄位是內建型別（**abap.char**）沒有掛 Data Element，Fiori 完全沒有標籤文字可用，要在 CDS View 欄位上直接加 **@EndUserText.label** 補上（**zi_rap03_umtest.ddls.abap / zi_rap03_umtest.ddlx.abap** 已同步更新）。
 - 另一個容易誤判的小地方：List Report 預設要求先按 **Go** 才會真正查詢，不按的話畫面固定顯示「To start, set the relevant filters.」——這跟資料庫是否有資料無關，之前幾次「畫面顯示 Tests (0)」都只是因為還沒按 **Go**，不是真的沒資料。
5. 如果你想額外用瀏覽器/Postman 自己測 **TestUnmanaged** 的 **Create**（走正常瀏覽器 Session，不會遇到上面的自我呼叫限制），記得要用外網對外主機名稱 + Port（**erpdemo01.itts.com.tw:44300**），服務網址是 [https://erpdemo01.itts.com.tw:44300/sap/opu/odata/sap/ZRAP04_SB/TestUnmanaged?\\$format=json](https://erpdemo01.itts.com.tw:44300/sap/opu/odata/sap/ZRAP04_SB/TestUnmanaged?$format=json)。

思考題

1. **TaskManaged** 這個 Entity Set 在 Fiori Elements 列表畫面能正常顯示資料，但按 **Create** 會直接 Dump——這個「讀取正常、寫入 Dump」的行為分界線，具體是卡在哪一層（CDS View？Service Definition？Service Binding？還是 Behavior Definition / Runtime）？（提示：回顧第 43 節 **CL_CSP_MD_METADATA_FACTORY** 的檢查邏輯，只卡在什麼時機點）
2. 一個 Service Definition 可以 **expose** 多個 CDS View，這一課刻意混搭了 Managed 跟 Unmanaged 兩種不同 Behavior 類型的實體到同一個服務裡——如果是你要設計一個真實的業務服務（例如訂單管理），什麼情況下你會想把多個「技術實作方式不同」的實體包進同一個服務？什麼情況下你反而會想拆成多個獨立服務？
3. **ZR_RAP04_SELFTEST** 用 **X-CSRF-Token: Fetch** 換 Token 之後，是靠同一個 **lo_client** 物件在後續請求沿用 Cookie / Session 狀態，才能讓拿到的 Token 生效。如果分別對每個請求都重新 **create_by_destination** 一次全新的 Client（而不是共用同一個），CSRF 驗證會不會失敗？為什麼？

答案

zrap04_sd.srvd.abap（Service Definition，Claude 建立）、**zr_rap04_selftest.prog.abap**（自我呼叫驗證程式，Claude 建立；讀取部分見驗證方式第 2 點已用 SE38 驗證成功，寫入部分見第 3 點無法用自我呼叫驗證）、**zrap04_sb.srvb.xml**（Service Binding，你在 Eclipse 建立 + Publish 後，Claude 用 ADT 讀回結構化內

容存成快照，`srvb:published="true"` 已確認）。Service Binding 的完整操作過程另外保留一份原始截圖記錄 `zrap04_ECLIPSE_SERVICE_BINDING_操作記錄.docx`。這一課收尾時順便補齊了 rap03 的 `zi_rap03_umtest.ddlx.abap`（新建）與 `zi_rap03_umtest.ddls.abap`（補上 `@Metadata.allowExtensions / @EndUserText.label`），讓 `TestUnmanaged` 的 Fiori Elements Create 畫面能正常顯示欄位標籤，並藉此完整驗證了 Unmanaged 路徑真的能端對端寫入成功。